Makshi Baskaran | MB10197 | N15988801

ChipChat Tutorial Assignment Report

ECE - GY 9953 Advanced Project

Abstract

This report documents the completion of the ChipChat tutorial assignment, which explores LLM-assisted hardware design through the generation and verification of synthesizable Verilog code. The assignment comprised three main components: solving two tutorial examples from the provided ChipChat notebook, documenting an iterative debugging process, and developing a prompt-engineering extension.

For Part I, two distinct hardware designs were successfully implemented and verified: a Binary-to-BCD Converter (combinational logic) and an 8-bit Shift Register (sequential logic). Both designs were generated using natural language prompts, compiled with iverilog, and validated against provided testbenches.

Part II focused on documenting the debugging and iteration process for the Shift Register design, demonstrating three distinct iterations involving workflow errors, initialization issues, and behavioral mismatches. Each iteration included specific diagnosis, fixes, and verification outcomes.

Part III extended the Shift Register design by adding a bidirectional shift capability through a direction control input. This extension required prompt refinement, new testbench development, and demonstrated advanced prompt-engineering techniques for controlling LLM-generated hardware specifications.

All designs successfully passed their respective testbenches, demonstrating the viability of LLM-assisted hardware design when combined with rigorous verification and iterative refinement.

1. Introduction

The ChipChat tutorial assignment explores the integration of Large Language Models (LLMs) into the hardware design workflow, specifically focusing on the automatic generation of synthesizable Verilog code from natural language specifications. This approach represents a paradigm shift in hardware design methodology, where designers can leverage AI to accelerate the initial RTL development phase while maintaining rigorous verification standards.

Traditional hardware design requires extensive knowledge of Hardware Description Languages (HDLs) and careful attention to synthesizability constraints, timing considerations, and design-for-test principles. By incorporating LLMs into this process, designers can potentially reduce the time from specification to implementation while maintaining code quality through automated testing and iterative refinement.

This assignment was structured to provide hands-on experience with three critical aspects of LLM-assisted hardware design: prompt engineering for hardware specifications, debugging and verification of generated code, and extending designs through iterative prompt refinement.

1.1 Assignment Objectives

- Generate synthesizable Verilog from natural language prompts using LLMs
- Extract and clean LLM-generated code for compilation with industry-standard tools
- Validate correctness through testbench-driven simulation using Icarus Verilog
- Practice iterative debugging based on compilation and simulation feedback
- Develop advanced prompt-engineering techniques for hardware design extensions

2. Tools and Methodology

2.1 Development Environment

Environment: Google Colab was used as the primary development platform, providing a cloud-based Jupyter notebook environment with pre-installed Python dependencies and easy integration with external APIs.

Programming Language: Python 3.x was used for notebook scripting, API interaction, and file manipulation.

Notebook Framework: Jupyter Notebook provided the interactive development environment for code generation, testing, and documentation.

2.2 Hardware Design and Verification Tools

Icarus Verilog (iverilog): An open-source Verilog compiler used for RTL compilation and elaboration. The compiler was invoked with the -g2012 flag to enable SystemVerilog 2012 features.

VVP (Verilog VVP): The Icarus Verilog runtime engine used to execute compiled simulations and generate test results.

GTKWave: Optional waveform viewer for debugging signal transitions and timing behavior.

2.3 LLM Configuration

Parameter	Value
Model	GPT-4 / DeepSeek (configurable)
Temperature	0.7 (balanced creativity and determinism)
Max Tokens	2048
API	OpenAI API (via openai Python package)

The temperature setting of 0.7 was chosen to balance creative problem-solving with consistent, deterministic output.

2.4 Workflow Methodology

The general workflow followed for each design was:

1. Specification Development: Create a detailed natural language prompt describing the hardware functionality
2. LLM Code Generation: Submit the prompt to the LLM API and capture the raw response
3. Code Extraction: Parse the LLM response to extract the Verilog module code
4. File Creation: Write the extracted Verilog to a .v file in the working directory
5. Compilation: Compile the design with the testbench using iverilog -g2012
6. Simulation: Execute the compiled testbench using vvp and analyze results

7. Debugging and Iteration: If errors occur, diagnose the issue, refine the prompt or code, and repeat

8. Verification: Confirm all test cases pass and document the final implementation

3. Part I: Tutorial Examples

This section presents the two tutorial examples completed for Part I of the assignment: a Binary-to-BCD Converter representing combinational logic design, and an 8-bit Shift Register representing sequential logic design.

3.1 Example A: Binary-to-BCD Converter

3.1.1 Design Overview

The Binary-to-BCD (Binary Coded Decimal) Converter is a combinational logic circuit that converts a 5-bit binary input (representing values 0-31) into an 8-bit BCD output consisting of two BCD digits: tens and ones. BCD encoding represents each decimal digit using 4 bits, where valid BCD values range from 0000 (0) to 1001 (9).

This converter is commonly used in digital systems that interface with seven-segment displays, decimal counters, or any application requiring decimal representation of binary values. The conversion algorithm employed is the double-dabble method.

3.1.2 Module Interface

Port Name	Direction	Width	Description
binary_input	Input	5 bits	Binary value to convert (0-31)
bcd_output	Output	8 bits	BCD representation [7:4]=tens, [3:0]=ones

3.1.3 Design Approach and Algorithm

The design implements the double-dabble algorithm, a well-known method for binary-to-BCD conversion. The algorithm works by iteratively shifting the binary number left while adjusting BCD digits that exceed 4 (binary 0100).

9. Initialize a shift register containing the binary input in the rightmost bits